arXiv:2402.04925v1 [cs.DC] 15 Jan 2024

TP-AWARE DEQUANTIZATION

Adnan Hoque
IBM T.J. Watson Research Center
Yorktown Heights, NY, United States
adnan.hoque1@ibm.com

Mudhakar Srivatsa
IBM T.J. Watson Research Center
Yorktown Heights, NY, United States
msrivats@us.ibm.com

Chih-Chieh Yang
IBM T.J. Watson Research Center
Yorktown Heights, NY, United States
chih.chieh.yang@ibm.com

Raghu Ganti
IBM T.J. Watson Research Center
Yorktown Heights, NY, United States
rganti@us.ibm.com

ABSTRACT

In this paper, we present a novel method that reduces model inference latency during distributed deployment of Large Language Models (LLMs). Our contribution is an optimized inference deployment scheme that address the current limitations of state-of-the-art quantization kernels when used in conjunction with Tensor Parallel (TP). Our method preserves data locality in GPU memory access patterns and exploits *a priori* knowledge of TP to reduce global communication. We demonstrate an up to 1.81x speedup over existing methods for Llama-70B and up to 1.78x speedup for IBM WatsonX's Granite-20B MLP layer problem sizes on A100 and H100 NVIDIA DGX Systems for a variety of TP settings.

***K*eywords** Deep Learning · Foundation Models · Quantization · Tensor Parallel · GPU · Optimization · LLM · Transformers · Distributed Systems

1 Introduction

Given the recent advancement of LLMs, deployment optimizations are becoming more crucial as the size of state-of-the-art LLMs increase in scale. As these these models continue to grow, so does the need to optimize the increasingly parallel and increasingly distributed workload requirements of modern-day deep learning inference. Strategies like GPTQ [1] and Tensor Parallel (TP) [4] are hence essential in achieving high-throughput performance. Our method is motivated by several key properties of GPTQ, TP and General Matrix Multiplication (GEMM). We build on these existing methods and present a key innovation that helps maximize memory throughput and reduce latency. Our method shows up to a 1.81x speedup on Llama-70B and up to a 1.78x speedup on Granite-20B MLP layer problem sizes. We achieve this by reducing global communication and enforcing data locality.

1.1 Motivation

We begin by introducing the grouping scheme in GPTQ-style quantization. The *group size* defines the quanta that the quantization strategy will be performed on. Thus, every *group size* number of input channels in the *K x N* weight matrix will share the same quantization metadata (scales and zeros). This introduces the need to spatially relate the weight matrix to the aforementioned metadata. This mapping must be remembered, as it will be used during deployment when performing *dequantization*. If we were to proceed with the basic formulation of GPTQ, the rows of the weight matrix are related to the metadata by a group index array. In this paper, we will refer to this basic formulation of the group index array as the *naive* approach and it can be evaluated as the following, where we let *G* equal the group size:

$$g_{\text{idx_naive}}[i] = \left\lfloor \frac{i}{\text{G}} \right\rfloor, \quad \text{for } i = 0, 1, 2, \ldots, K-1 \tag{1}$$

Notably, this approach does not consider further optimizations introduced by the GPTQ authors. Our method targets the complexities that arrive when considering the "Activation Order" optimization. This parameter was introduced by the authors to improve model accuracy [2] and is triggered through an optional flag act_order in the GPTQ package. When this flag is set to True, the quantization process will behave differently. This optimization reduces overall quantization error by processing the rows of the weight matrix in a way that respects the significance of the weights impacts on model accuracy. Thus, the weights with higher impact on accuracy will incur less quantization error, and vice versa for weights with lower impact on accuracy. The implication is then, with act_order (also known as desc_act in different packages), the rows of the weight matrix are reordered based on the properties discussed above and notably, the new ordering *must* be respected when performing inference during dequantization for each row of the weight matrix and it's corresponding metadata. This transformation is realized through the group index array. Here, we use a random permutation function ϕ to emulate an arbitrary reordering. The new group index array can then be expressed by the following expression:

$$\text{Let } \phi : \{0, 1, \ldots, K-1\} \rightarrow \{0, 1, \ldots, K-1\} \text{ be a random permutation function.} \tag{2}$$

$$g_{\text{idx_actorder}}[i] = \left\lfloor \frac{\phi(i)}{\text{G}} \right\rfloor, \quad \text{for } i = 0, 1, 2, \ldots, K-1 \tag{3}$$

This approach however, introduces performance overhead due to the sub-optimal memory access pattern of having to frequently reload quantization metadata [2] due to the *unordered* nature of the group index array. Further, when this optimization is used in a TP setting this method incurs communication overhead, a detail we expand on in the next section. Our method is then, a way to reap the benefits of the accuracy boost provided by the act_order optimization without reducing memory throughput and incurring additional communication overhead.

2 Method

2.1 Enforcing Data Locality

Popular software packages that implement the GPTQ algorithm [7] will store the weights of the model on disk without including knowledge of the ordering suggested by Equation 3. Thus, we have a choice to use the mapping in Equation 3 to correctly map rows of the weight matrix to it's corresponding metadata as seen in Figure 1 during model deployment. As discussed previously, this leads to a sub-optimal memory access pattern. The method described in this section, is used in the state-of-the-art ExllamaV2 [6] kernel to enforce data locality by transforming the unordered group index array resulting from the act_order flag. To illustrate this method, we define a routine that takes as input the group index array, and returns an *ordered* group index array, $g_{idx_optimized}$ as well as it's associated permutation array *P*, using the torch.argsort [5] function.

Algorithm 1 Reorder Function

```
function REORDER(g_idx_actorder)                ▷ g_idx_actorder is a 1D array containing unordered indices
    P ← ARGSORT(g_idx_actorder)                 ▷ P is a 1D array containing the permutation
    g_idx_optimized ← g_idx_actorder[P]         ▷ Permute g_idx_actorder
    return P, g_idx_optimized                   ▷ g_idx_optimized is a 1D array containing ordered indices
end function
```

The optimized group index array has the desirable property of enforcing data locality during model inference. Consider Figure 1 and Figure 2. In Figure 2 we see that for multiple rows of the weight matrix *W* we are able to reuse the associated metadata, as the indices have been sorted such that it *guarantees* all members of the the same group are consecutive in *W*.

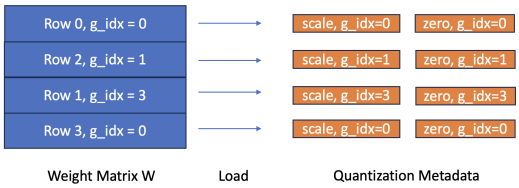

Figure 1: Naive Load with Activation Order Flag

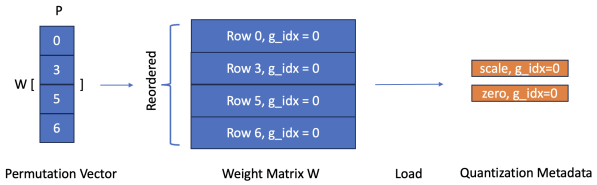

Figure 2: Optimized Load with Activation Order Flag

The optimized loading scheme increases data locality and memory throughput. The reordering of the weight matrices can be performed offline, and thus the permutation array *P* along with the optimized group index array are available to the model inference pipeline as global variables. If we will be deploying with TP model parallelism these parameters must be passed and sharded to our input activations *X*, model weights *W* and quantization metadata as they will be used to perform the correct mapping during dequantization. However, with no further optimization, this strategy that we will refer to as the *Naive Algorithm*, will incur an extra AllGather communication necessitated by the observation that the output of the Column-TP layer in Figure 4 will require reordering, as seen in Line 2 of Algorithm 2, to produce a correctly aligned tensor for the subsequent GEMM operation.

This leads us to our main contribution, which is a method to reduce global communication and thus reduce latency during model inference.

2.2 Minimizing Global Communication

In Transformer based architectures, we typically observe that popular implementations consist of 2 linear layers in the Attention block and 2 linear layers in the Multilayer Perceptron Layer (MLP) block as depicted in Figure 3. We follow the same model parallel strategy outlined in Megatron-LM [3]. This is to say, our method assumes the interleaved pattern of Column-TP with Row-TP as seen in Figure 4 for consecutive linear layers. In dequantization, we now have two choices. If we proceed with the optimized formulation of GPTQ with act_order=True, we must pass either the group index array outlined in Equation 3 or the more optimal cache-friendly group index array from Algorithm 1, used by the ExllamaV2 kernel.

If we proceed with the latter, during model inference, an expensive AllGather communication is required before the output shards of the first linear layer *Y1* are passed to the sharded weights of the second linear layer. Our insight, is then a method that avoids this communication using the intrinsic properties of GEMM.

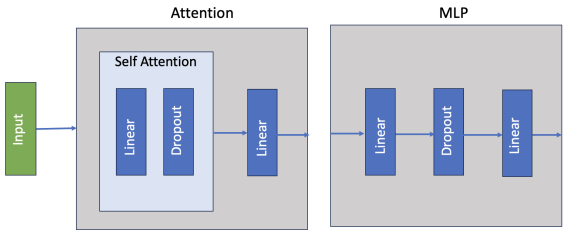

Figure 3: Transformer Block

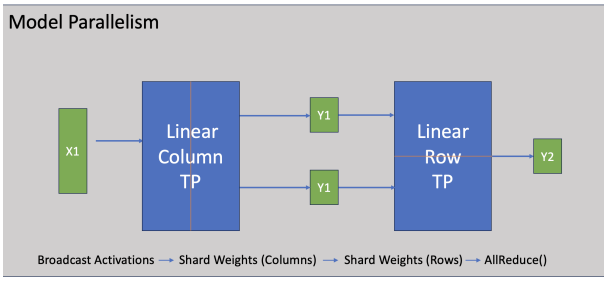

Figure 4: TP-Aware Model Parallelism

To motivate our method, we examine the current approach with the optimized group index array used by ExllamaV2. We first note, the interleaved Column-TP and Row-TP depicted in Figure 4 will produce two sharded weight matrices, $W1$, which is split column-wise across N ranks, and $W2$ which is split row-wise across N ranks. During dequantization, we will produce two permutation arrays, $P1$ and $P2$. Recall these arrays are optimized in that they enforce data locality when they are used to reorder the weights and load quantization metadata (scales and zeros). Notably, these two permutation arrays are available globally and can be computed offline.

We define Algorithm 2, and call it the *Naive Algorithm* to illustrate the above approach. We assume, activations, $X1$, the weight matrices $W1$ and $W2$ and permutation arrays $P1$ and $P2$ are available as input to the model. For a given matrix M, we use the following notation $M[P1, P2]$ to denote row and column permutations.

Noting the global communication required in Line 2 of Algorithm 2, we present a reordering strategy that avoids this communication across ranks and hence yields impressive latency improvements. We achieve this by by re-ordering the columns of $W1$ with the permutation array $P2$. This optimization is realized from the following insight.

By permuting the columns of $W1$ with $P2$ and then proceeding with the GEMM as is done Line 1 of Algorithm 3 we are aligning $Y1$ in such a way it no longer needs to be permuted with $P2$ in the subsequent GEMM with $W2$. Producing this alignment, is our key contribution, as it makes it possible to avoid global communication after the column-TP layer. Note that our method as it stands, only applies to the MLP layers of the Transformer block. This is due to the fact that the sharding strategy for Attention when employing MHA (Multi-Headed Attention), MQA (Multi-Query Attention) or GQA (Group-Query Attention) motivates the need for additional tricks to avoid global communication in TP, when the weights are reordered. We call our approach that applies to the MLP layers in the Transformer Block the *TP-Aware Algorithm*.

Algorithm 2 Naive Algorithm

Require: $X1, W1[P1], W2[P2]$ ▷ Input activations $X1$ and sharded reordered weight matrices $W1$ and $W2$
Require: $P1, P2$ ▷ Permutation arrays
Require: $rank, size$ ▷ Processor rank and total number of processors
1: $Y1_{local} \leftarrow X1_{global}[:, P1]$ @ $W1_{local}$ ▷ GEMM
2: $Y1_{global} \leftarrow \text{ALLGATHER}(Y1_{local})$ ▷ Gather $Y1$ shards from all processors
3: $Y1_{global} \leftarrow Y1_{global}[:, P2]$ ▷ Y1 reordered globally
4: $Y1_{local} \leftarrow \text{CHUNK}(Y1_{global}, rank, size, dim = 1)$ ▷ Shard $Y1$ among processors
5: $Y2_{local} \leftarrow Y1_{local}$ @ $W2_{local}$ ▷ GEMM with gathered $Y1$
6: $Y2_{global} \leftarrow \text{ALLREDUCE}(Y2_{local}, op = SUM)$ ▷ Reduce $Y2$ shards across all processors
7: **return** $Y2_{global}$ ▷ Each processor has the globally reduced result

Algorithm 3 TP-Aware Algorithm

Require: $X1, W1[P1, P2], W2[P2]$ ▷ Input activations $X1$ and sharded reordered weight matrices $W1$ and $W2$
Require: $P1$ ▷ Permutation array
Require: $rank, size$ ▷ Processor rank and total number of processors
1: $Y1_{local} \leftarrow X1_{global}[:, P1]$ @ $W1_{local}$ ▷ GEMM
2: $Y2_{local} \leftarrow Y1_{local}$ @ $W2_{local}$ ▷ GEMM with local $Y1$
3: $Y2_{global} \leftarrow \text{ALLREDUCE}(Y2_{local}, op = SUM)$ ▷ Reduce $Y2$ shards across all processors
4: **return** $Y2_{global}$ ▷ Each processor has the globally reduced result

3 Experiments

We conducted our experiments on a variety of TP settings and batch sizes, on highly relevant Llama-70B and Granite-20B, the flagship offering in IBM WatsonX, MLP layer sizes. We tested on enterprise A100x8 with Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60 and H100x8 with Intel(R) Xeon(R) Platinum 8480 CPU @ 3.00 GHz NVIDIA DGX systems. As our algorithm avoids communication in-between column-TP and row-TP layers we use FP16 to demonstrate this benefit. For our experimental results we denote M as the batch size, K1 and N1 as the input and output features respectively of the column-TP layer and N2 as the input features of the row-TP layer and have enabled TP using 1,2,4,8 GPUs in one node. As a simplification for the analysis, in the Llama-70B test case we assume single up_{proj} layer followed by $down_{proj}$ which allows us to directly compare with Granite's MLP layer. Our method can be generalized to the implementation in practice where a $gate_{proj}$ layer is also present.

4 Llama-70B

Latency Llama-70B

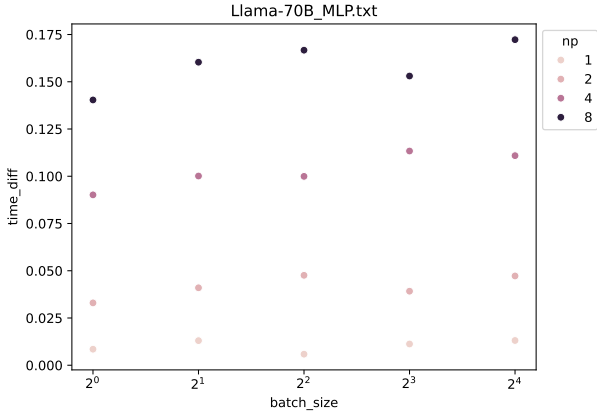

Figure 5: Latency Difference for Llama-70B, A100

Speedup Llama-70B

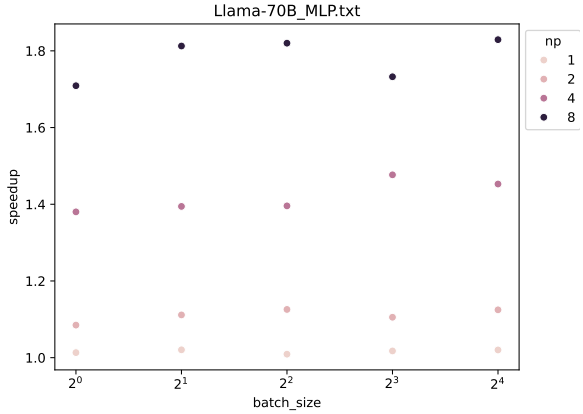

Figure 6: Speedup for Llama-70B, A100

4.1 Baseline TP=1

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)
1	(8192, 28672, 8192)	0.696	0.688
2	(8192, 28672, 8192)	0.694	0.683
4	(8192, 28672, 8192)	0.685	0.678
8	(8192, 28672, 8192)	0.706	0.697
16	(8192, 28672, 8192)	0.710	0.695

Table 1: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Llama-70B, TP=1, A100)

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)
1	(8192, 28672, 8192)	0.489	0.481
2	(8192, 28672, 8192)	0.471	0.466
4	(8192, 28672, 8192)	0.474	0.468
8	(8192, 28672, 8192)	0.471	0.464
16	(8192, 28672, 8192)	0.474	0.468

Table 2: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Llama-70B, TP=1, H100)

4.2 TP=2

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(8192, 28672, 8192)	0.493	0.433	1.14x
2	(8192, 28672, 8192)	0.508	0.407	1.25x
4	(8192, 28672, 8192)	0.519	0.412	1.26x
8	(8192, 28672, 8192)	0.516	0.418	1.23x
16	(8192, 28672, 8192)	0.501	0.416	1.20x

Table 3: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Llama-70B, TP=2, A100)

Average Speedup
1.22x

Table 4: Average Speedup for Llama-70B, TP=2, A100

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(8192, 28672, 8192)	0.302	0.283	1.07x
2	(8192, 28672, 8192)	0.316	0.285	1.11x
4	(8192, 28672, 8192)	0.323	0.286	1.13x
8	(8192, 28672, 8192)	0.320	0.289	1.11x
16	(8192, 28672, 8192)	0.322	0.289	1.11x

Table 5: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Llama-70B, TP=2, H100)

Average Speedup
1.11x

Table 6: Average Speedup for Llama-70B, TP=2, H100

4.3 TP=4

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(8192, 28672, 8192)	0.472	0.282	1.67x
2	(8192, 28672, 8192)	0.512	0.286	1.79x
4	(8192, 28672, 8192)	0.513	0.287	1.79x
8	(8192, 28672, 8192)	0.518	0.285	1.82x
16	(8192, 28672, 8192)	0.512	0.286	1.79x

Table 7: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Llama-70B, TP=4, A100)

Average Speedup
1.78x

Table 8: Average Speedup for Llama-70B, TP=4, A100

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(8192, 28672, 8192)	0.258	0.192	1.34x
2	(8192, 28672, 8192)	0.275	0.192	1.43x
4	(8192, 28672, 8192)	0.273	0.193	1.41x
8	(8192, 28672, 8192)	0.278	0.197	1.41x
16	(8192, 28672, 8192)	0.281	0.198	1.42x

Table 9: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Llama-70B, TP=4, H100)

Average Speedup
1.40x

Table 10: Average Speedup for Llama-70B, TP=4, H100

4.4 TP=8

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(8192, 28672, 8192)	0.495	0.284	1.74x
2	(8192, 28672, 8192)	0.503	0.276	1.82x
4	(8192, 28672, 8192)	0.539	0.291	1.85x
8	(8192, 28672, 8192)	0.530	0.286	1.85x
16	(8192, 28672, 8192)	0.512	0.286	1.79x

Table 11: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Llama-70B, TP=8, A100)

Average Speedup
1.81x

Table 12: Average Speedup for Llama-70B, TP=8, A100

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(8192, 28672, 8192)	0.245	0.144	1.70x
2	(8192, 28672, 8192)	0.256	0.146	1.75x
4	(8192, 28672, 8192)	0.257	0.144	1.78x
8	(8192, 28672, 8192)	0.258	0.145	1.78x
16	(8192, 28672, 8192)	0.266	0.149	1.78x

Table 13: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Llama-70B, TP=8, H100)

Average Speedup
1.76x

Table 14: Average Speedup for Llama-70B, TP=8, H100

5 Granite-20B

Latency Granite-20B

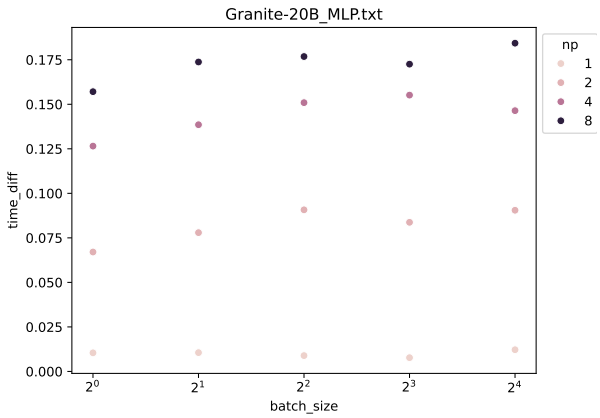

Figure 7: Latency Difference for Granite-20B, A100

Speedup Granite-20B

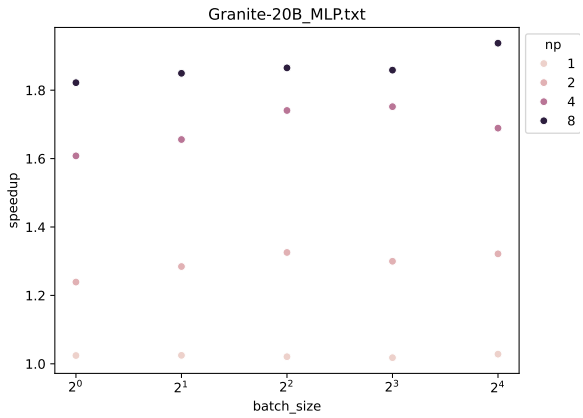

Figure 8: Speedup for Granite-20B, A100

5.1 Baseline TP=1

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)
1	(6144, 24576, 6144)	0.482	0.474
2	(6144, 24576, 6144)	0.476	0.471
4	(6144, 24576, 6144)	0.482	0.469
8	(6144, 24576, 6144)	0.479	0.467
16	(6144, 24576, 6144)	0.487	0.475

Table 15: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Granite-20B, TP=1)

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)
1	(6144, 24576, 6144)	0.349	0.341
2	(6144, 24576, 6144)	0.335	0.328
4	(6144, 24576, 6144)	0.325	0.319
8	(6144, 24576, 6144)	0.335	0.327
16	(6144, 24576, 6144)	0.335	0.328

Table 16: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Granite-20B, TP=1, H100)

5.2 TP=2

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(6144, 24576, 6144)	0.486	0.309	1.57x
2	(6144, 24576, 6144)	0.476	0.471	1.01x
4	(6144, 24576, 6144)	0.482	0.469	1.03x
8	(6144, 24576, 6144)	0.479	0.467	1.03x
16	(6144, 24576, 6144)	0.504	0.306	1.65x

Table 17: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Granite-20B, TP=2, A100)

Average Speedup
1.26x

Table 18: Average Speedup for Granite-20B, TP=2, A100

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(6144, 24576, 6144)	0.263	0.214	1.23x
2	(6144, 24576, 6144)	0.279	0.218	1.28x
4	(6144, 24576, 6144)	0.284	0.220	1.29x
8	(6144, 24576, 6144)	0.285	0.220	1.29x
16	(6144, 24576, 6144)	0.285	0.221	1.29x

Table 19: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Granite-20B, TP=2, H100)

Average Speedup
1.28x

Table 20: Average Speedup for Granite-20B, TP=2, H100

5.3 TP=4

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(6144, 24576, 6144)	0.500	0.292	1.71x
2	(6144, 24576, 6144)	0.497	0.284	1.75x
4	(6144, 24576, 6144)	0.518	0.293	1.77x
8	(6144, 24576, 6144)	0.508	0.284	1.79x
16	(6144, 24576, 6144)	0.530	0.290	1.83x

Table 21: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Granite-20B, TP=4, A100)

Average Speedup
1.77x

Table 22: Average Speedup for Granite-20B, TP=4, A100

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(6144, 24576, 6144)	0.251	0.156	1.61x
2	(6144, 24576, 6144)	0.267	0.157	1.70x
4	(6144, 24576, 6144)	0.268	0.158	1.70x
8	(6144, 24576, 6144)	0.269	0.159	1.69x
16	(6144, 24576, 6144)	0.269	0.159	1.69x

Table 23: Naive and TP Aware Model Algorithm for Various Batch Sizes (Granite-20B, TP=4, H100)

Average Speedup
1.68x

Table 24: Average Speedup for Granite-20B, TP=4, H100

5.4 TP=8

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(6144, 24576, 6144)	0.512	0.294	1.74x
2	(6144, 24576, 6144)	0.530	0.291	1.82x
4	(6144, 24576, 6144)	0.537	0.293	1.83x
8	(6144, 24576, 6144)	0.541	0.305	1.77x
16	(6144, 24576, 6144)	0.551	0.303	1.82x

Table 25: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Granite-20B, TP=8, A100)

Average Speedup
1.80x

Table 26: Average Speedup for Granite-20B, TP=8, A100

M	K1, N1, N2	Naive Algorithm (ms)	TP Aware Algorithm (ms)	Speedup
1	(6144, 24576, 6144)	0.252	0.148	1.70x
2	(6144, 24576, 6144)	0.255	0.142	1.79x
4	(6144, 24576, 6144)	0.259	0.141	1.84x
8	(6144, 24576, 6144)	0.257	0.140	1.84x
16	(6144, 24576, 6144)	0.255	0.140	1.82x

Table 27: Naive and TP Aware Algorithm Latencies for Various Batch Sizes (Granite-20B, TP=8, H100)

Average Speedup
1.78x

Table 28: Average Speedup for Granite-20B, TP=8, H100

For Llama-70B problem sizes, our TP-Aware method demonstrates an average speedup of 1.22x, 1.78x and 1.81x for TP = 2, 4, 8 respectively on the 8xA100 system. On the 8xH100 system our method demonstrated an average speedup of 1.11x, 1.40x and 1.76x. For Granite-20B problem sizes, we achieve an average speedup of 1.26x, 1.77x and 1.80x on the A100 and 1.28x, 1.68x and 1.78x on the H100.

Notably, as the number of ranks increased so did the corresponding performance improvement. This is the expected result as we expect the naive algorithm communication overhead to increase relatively to overall cost when number of ranks increases. This demonstrates our methods superior scalability over existing methods.

6 Conclusion

In this paper we present a novel insight, and discussed it's potential impacts given the increasingly distributed workload of modern day LLM inference. Our contribution is an optimized reordering strategy that minimizes global communication across ranks. We call our method TP-Aware Dequantization. We benchmarked our method on Llama-70B and Granite-20B problem sizes and demonstrated up to 1.81x and 1.76x speedups, respectively, on the NVIDIA DGX A100 and an up to 1.76x and 1.78x speedup on the NVIDIA DGX H100.

7 Acknowledgements

We would like to acknowledge important contributions from Less Wright, Meta AI, for providing the H100 experiment results, Hao Yu and Eun Kyung Lee from IBM Research for their review on the presented materiel.

References

[1] Elias Frantar et al. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. Mar. 22, 2023. arXiv: 2210.17323[cs]. URL: http://arxiv.org/abs/2210.17323 (visited on 12/26/2023).

[2] Maxime Lbonne. *ML Blog - 4-bit LLM Quantization with GPTQ*. July 30, 2023. URL: https://mlabonne.github.io/blog/posts/4_bit_Quantization_with_GPTQ.html (visited on 12/26/2023).

[3] Deepak Narayanan et al. *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*. Aug. 23, 2021. arXiv: 2104.04473[cs]. URL: http://arxiv.org/abs/2104.04473 (visited on 12/26/2023).

[4] *Tensor Parallelism - torch.distributed.tensor.parallel — PyTorch 2.1 documentation*. URL: https://pytorch.org/docs/stable/distributed.tensor.parallel.html (visited on 12/21/2023).

[5] *torch.argsort — PyTorch 2.1 documentation*. URL: https://pytorch.org/docs/stable/generated/torch.argsort.html (visited on 01/04/2024).

[6] turboderp. *turboderp/exllamav2*. original-date: 2023-08-30T08:54:22Z. Dec. 28, 2023. URL: https://github.com/turboderp/exllamav2 (visited on 12/28/2023).

[7] William. *PanQiWei/AutoGPTQ*. original-date: 2023-04-13T02:18:11Z. Dec. 27, 2023. URL: https://github.com/PanQiWei/AutoGPTQ (visited on 12/27/2023).